Unidad 4
Actividad Práctica

Integrantes:
Hakanson, Ian
Morales, Elias

PARTE-2:

Esta parte será coevaluada en la Segunda Instancia de evaluación el día 27 de octubre .

1. Seleccionar una funcionalidad que funcione y este completa.
   RF-AL-01: El sistema debe permitir al Alumno consultar su información personal y académica (Legajo, DNI, Nombre, Apellido, Fecha de nacimiento, Domicilio, Nivel educativo, Curso y Estado).
2. De acuerdo al análisis del contenido de la unidad 4, completar la siguiente tabla:

Contenido	**Aplicado**		**No Aplicado**
	Ejemplificar con las herramientas y/o estrategias aplicadas según corresponda	Ejemplificar un flujo	
Integración Continua	GitHub Actions (.github/workflows/ci.yml): en cada push y Pull Request a main se ejecutan checkout, Node 20, npm install, typecheck, los 50 tests (Vitest + pg-mem + supertest) y build de API y web. Cada ejecución queda registrada en la pestaña Actions.	—	
Despliegue Continuo	Render toma el último commit de main y despliega automáticamente la API y el frontend	—	

	estático definidos en render.yaml; PostgreSQL se provisiona como base gestionada y scripts/init-db.mjs inicializa el esquema en el primer arranque. Con autoDeployTrigger: checksPass, el deploy espera a que CI termine correctamente.		
Build	Automatizado en CI: tsc para la API, tsc + Vite para la web y typecheck del paquete móvil. Los Dockerfiles empaquetan API y web; el pipeline publica apps/api/dist y apps/web/dist. Render ejecuta npm install + build en su entorno para API y web.	—	
Pipelines	Pipeline build-and-test en GitHub Actions con etapas encadenadas: checkout → setup Node → install → typecheck → tests → build → upload-artifact. Se dispara por push o PR contra main.	—	
Variables de Entorno	apps/api/.env.example y config.ts centralizan DATABASE_URL, JWT_SECRET, PORT, CORS_ORIGIN, APP_URL, UPLOADS_DIR, SMTP_* y BANK_*. Podman/Compose las inyectan; Render define	—	

	DATABASE_URL, JWT_SECRET, CORS_ORIGIN, APP_URL y VITE_API_URL; la app móvil usa EXPO_PUBLIC_API_URL. Los secretos no se versionan.		
Gestión de Artefactos	GitHub Actions publica apps/api/dist y apps/web/dist con actions/upload-artifact (retención 7 días). Las imágenes Docker se construyen localmente; publicarlas en un registry queda pendiente. Render reconstruye desde el código fuente; los artefactos de CI quedan para trazabilidad y reutilización.	—	
Integración de repositorios con herramientas CI/CD	GitHub Actions está conectado al repositorio: cada push/PR a main genera una corrida con resultado y artefactos. Trazabilidad commit → workflow run → resultado → artefacto.	—	
Pull Requests y Merge Requests	Existe .github/pull_request_template.md y se mergearon 2 Pull Requests (#1 y #2) desde rama_nueva. Falta activar branch	—	

	protection con el check build-and-test como obligatorio. Cada merge a main dispara el auto-deploy de Render.		
Issues		No se usaron issues: el seguimiento se hizo con commits, Pull Requests y documentación en docs/.	X
Wikis		La Wiki de GitHub está habilitada pero vacía; la documentación vive como archivos Markdown/PDF en docs/.	X
Tablero Kanban		No hay tablero en GitHub Projects; la planificación está en docs/planificacionDeSprints.md y docs/planDeTrabajo*.md, sin visualización To Do / In Progress / Done.	X
Otro/s	Pruebas automatizadas (Vitest + pg-mem + supertest, 50 tests); typecheck de API, web y móvil; patrones de diseño GoF (Singleton, Proxy, Iterator); app móvil Expo y módulo económico de cuotas, facturas, comprobantes y pagos. Despliegue automático en Render con blueprint (render.yaml).	npm test → Vitest levanta una base pg-mem en memoria y corre 50 tests de autenticación, RBAC, inscripciones, transporte, comedor, reportes, facturación y comprobantes. Se ejecuta a demanda y también dentro del job build-and-test de GitHub Actions.	

3. Contenidos NO APLICADOS,
    a. Seleccionar dos opciones que a su criterio deben aplicarse para optimizar el desarrollo de la aplicación.
    b. Realizar la aplicación correspondiente.
    c. Entrega: 27 de octubre para su evaluación

RESPUESTA:

a. Selección: se eligieron Integración Continua (CI) y Pull Requests y Merge Requests. Son las dos palancas de mayor impacto sobre el resto de los contenidos

no aplicados: la CI resuelve a la vez Build automatizado, Pipelines e Integración del repositorio con una herramienta de CI/CD; y los Pull Requests son el mecanismo que dispara esa CI como quality gate antes de llegar a main, además de habilitar code review.

b. Aplicación realizada:

- Pipeline de Integración Continua (.github/workflows/ci.yml): workflow de GitHub Actions que se dispara en cada push y cada Pull Request contra main. Ejecuta, en orden: checkout del código, configuración de Node 20, instalación de dependencias, typecheck de API/web/móvil, ejecución de los 50 tests (Vitest + pg-mem + supertest) y build (tsc + Vite). Al finalizar, publica como artefacto los directorios apps/api/dist y apps/web/dist. Después del merge, Render toma el último commit de main y despliega API y web.

- Validación local antes de subir el pipeline: npm run typecheck sin errores; npm test con 50/50 tests pasados (autenticación, RBAC, inscripciones, transporte, comedor, reportes, HU-05, HU-07, facturación y comprobantes); npm run build genera apps/api/dist y apps/web/dist correctamente. Tiempo total del pipeline (typecheck + test + build): 33,1 segundos.

- Flujo de Pull Requests y Merge Requests (.github/pull_request_template.md): plantilla con descripción del cambio, tipo de cambio, checklist de calidad (CI en verde, tests agregados, sin secretos en el código, rama actualizada) y un apartado para describir cómo probarlo. El flujo de trabajo pasa a ser: Issue → rama feature/* → commits → Pull Request → CI (build-and-test) → Code Review → Merge a main. Ya se mergearon 2 Pull Requests (#1 y #2) desde rama_nueva. Cada merge a main dispara el auto-deploy de Render.

- Para que este flujo sea obligatorio falta activar en GitHub: Settings → Branches → Branch protection rules → Require status checks to pass before merging, seleccionando el check build-and-test (requiere permisos de administrador del repositorio). Además, render.yaml puede declarar autoDeployTrigger: checksPass para que Render espere a que CI termine antes de desplegar.

4. Métricas de productividad
    a. Seleccionar herramientas de métrica que permitan medir el rendimiento de la aplicación con respecto a velocidad de entrega,cuellos de botella y oportunidades de mejora.

RESPUESTA:

- Script propio sobre el historial de git (scripts/metrics/git_metrics.py): calcula Deployment Frequency y Lead Time (proxy) a partir de los commits reales de main, sin depender de la API de GitHub.

- GitHub Actions (una vez subido el pipeline de CI): tiempo de ejecución de cada job y tasa de éxito/fallo, visibles en la pestaña Actions del repositorio.

- GitHub Insights/Pulse y la API de Pull Requests (created_at / merged_at): para medir, a partir de ahora, el Lead Time real como tiempo entre apertura y merge de cada PR.

b. Aplicar las herramientas y mostrar los resultados obtenidos.

RESPUESTA:

Se ejecutó git_metrics.py sobre el historial real de la rama main del repositorio (32 commits, del 23/08/2026 al 27/10/2026): Con el auto-deploy de Render, cada commit/merge a main genera un despliegue.

- Deployment Frequency (commits/merges a main; con auto-deploy de Render, cada uno genera un despliegue): 3,45 despliegues/semana.

- Lead Time promedio entre integraciones: 50,3 horas.

- Lead Time mediana: 0,2 horas.

- Lead Time máximo: 450,4 horas (≈18,8 días).

- Lead Time mínimo: 0,0 horas.

Adicionalmente, al correr localmente el pipeline de CI implementado en el punto 3, se midió un tiempo de 33,1 segundos para validar un cambio completo (typecheck + 50 tests + build), frente a la validación manual previa, que no tenía un tiempo definido y no dejaba registro sistemático (0 corridas de CI antes de incorporar el workflow).

c. En función de los resultados obtenidos, especificar estrategias de mejoras,

RESPUESTA:

La mediana (0,2 h) y el máximo (450,4 h) muestran una distribución muy irregular: muchas integraciones se agruparon en pocas horas y hubo un período prolongado sin integraciones. Además, antes de incorporar la CI el 100% de esas integraciones se hicieron sin validación automática, por lo que el riesgo de que un error llegue a main sin detectarse era alto. Estrategias definidas:

- Automatizar la validación con CI (ya implementado): cada integración se compila y prueba en 33,1 s en GitHub Actions, en lugar de depender de una revisión manual indeterminada.

- Exigir Pull Requests con CI en verde como quality gate antes de mergear a main (flujo adoptado; falta activar branch protection).

- Fomentar integraciones más chicas y frecuentes, para reducir el Lead Time y su variabilidad.

- Medir el Lead Time real como tiempo entre apertura y merge del Pull Request (más preciso que el proxy basado en commits), usando la API de Pull Requests de GitHub cuando el equipo esté autenticado.

- Publicar las imágenes Docker en un registry para completar la gestión de artefactos y reutilizar la misma versión en cada despliegue.

- Configurar Render con autoDeployTrigger: checksPass para que el despliegue del último commit espere a que CI termine, y automatizar las migraciones sobre bases existentes (hoy scripts/init-db.mjs solo inicializa el esquema la primera vez).

d. Aplicar las estrategias de mejoras y mostrar los resultados obtenidos mediante una tabla comparativa.

RESPUESTA:

Las estrategias de CI y Pull Requests ya están implementadas y en uso: el workflow se ejecuta en GitHub Actions, se publican artefactos y se mergearon 2 Pull Requests. La columna “Después” refleja lo que el pipeline mide y garantiza; el Lead Time y la Deployment Frequency reales post-adopción (medidos con la API de Pull Requests) deberán actualizarse cuando el equipo use el flujo durante algunas semanas. También queda pendiente activar branch protection con el check build-and-test como obligatorio. Render despliega automáticamente el último commit de main; queda pendiente que las migraciones se apliquen en cada deploy sobre una base ya inicializada.

Métrica	**Antes (medido sobre el historial real del repo)**	**Después (con CI + Pull Requests implementados)**
Deployment Frequency (commits/merges a main; auto-deploy de Render)	3,45 commits/semana (32 commits en 65 días, del 23/08/2026 al 27/10/2026); con auto-deploy, equivalente a ~3,45 despliegues/semana.	Render despliega automáticamente el último commit de main cuando CI pasa; la frecuencia real de despliegues acompaña la de merges a main.
Lead Time entre integraciones — mediana	0,2 horas	El tiempo de validación pasa a 33,1 s de pipeline automatizado (typecheck + 50 tests + build); con autoDeployTrigger: checksPass, Render espera esa validación antes de desplegar.
Lead Time entre integraciones — máximo	450,4 horas (≈18,8 días)	Acotado por el quality gate del PR: ningún cambio puede acumularse sin validar más allá de lo que tarda en abrirse y revisarse el Pull Request.
Cobertura de validación automática antes de llegar a main	0 % (ninguna corrida de CI antes de incorporar el workflow)	100 % desde la incorporación: todo push o PR contra main dispara typecheck + 50 tests + build automáticamente (job build-and-test de GitHub Actions); Render despliega el último commit solo si CI pasa.
Trazabilidad cambio → resultado de validación	Ninguna: no hay registro de qué se probó en cada commit	Cada commit/PR queda asociado a una corrida de GitHub Actions y cada deploy de Render queda asociado al commit desplegado; los artefactos de build se publican.

e. Fecha de entrega: 27 de octubre.